

Muhammad Farooqi — Interview Prep Cheat Sheet

Rivian — Navigation Software Engineering Intern

Q1: Driver Display Swap Feature & Test Cases

Interviewer question:

Walk me through what the Driver Display swap feature actually was — what was it doing, why did it need 40 test cases, and what did one of those 5 critical blockers actually look like in practice?

Model Answer:

The Driver Display swap feature lets the driver toggle between Maps mode and ADAS mode on the ICD (Instrument Cluster Display) using the right-side steering buttons. Maps mode shows turn-by-turn navigation; ADAS mode shows surrounding vehicles and lane assist overlays. As the navigation team's owner of this feature, I created the 40 test cases covering the maps-specific scenarios: correct turn maneuvers, Lane Keep Assist rendering, lane departure warnings, and map state transitions. The initial test cases were manual — a person walked through each scenario against the test plan.

The 5 critical blockers we caught included: incorrect turn maneuver icons, missing road names on the map, and a couple of visual misalignments on the ICD. We caught these during regression testing before the feature shipped, filed them, and had them fixed before release.

On the automation side: we started exploring Appium to automate the center display, using screenshot comparisons against golden images to catch visual regressions. The ICD has no touch capability which added complexity. We later moved toward using the official Appium MCP with a cursor CLI in script form — giving developers a way to self-test before anything reaches QA, so QA focuses on real end-to-end scenarios rather than trivial visual bugs.

Q2: Downstream CI Pipeline & Early Merge-Request Checks

Interviewer question:

Walk me through exactly what was happening before you built this, what you actually built technically, and why triggering jobs in a separate test repository was the right solution rather than just adding checks to the main repo.

Model Answer:

Before this, developers had no way to validate their changes against the integration test suite before merging. The automated tests lived in a separate repo from the main codebase, and building the full Android layer on an XMM took about 3 hours — and not every dev even had a physical bench to test on. So 10-20 MRs would accumulate daily and get sent to the integration team all at once. When a build broke, the team had to bisect across all those MRs to find the culprit, which happened roughly every couple of days.

What I built was a pipeline that triggered the downstream integration test jobs from the separate repo directly on a merge request, before it ever hit the daily build. So instead of finding breakages after the fact, devs got signal early — on a free runner, without needing a physical bench or a 3-hour build.

■ *The 10-20% figure refers to the rough frequency of build-breaking MRs observed. If pressed, frame it as 'roughly every couple of days out of daily builds' — don't overstate a metric you can't defend precisely.*

Q3: Nav API Tests — Reroutes & Trip Refresh

Interviewer question:

What does a nav API test actually look like in practice — what were you calling, what were you asserting, and why did reroutes and trip refresh need to be tested across three different vehicle models?

Model Answer:

These were API-level tests against Rivian's internal navigation framework. The flow was: authenticate first using the internal auth layer, then hit the navigation endpoints — POST endpoints for passing in lat/long coordinates, and GET endpoints for fetching route data from ABRP (the route planning service Rivian uses).

To simulate a reroute, I used a separate POST endpoint to mock the vehicle's GPS location — programmatically moving the car off-route without physically driving it — then asserted that a reroute was triggered. For faster route tests, I'd set a destination, lock in a slower route, then verify the API surfaced a faster alternative with the correct time delta. The R2-specific tests covered weather-based destination features releasing on R2 before R1, so those needed to be validated in isolation against R2 endpoints.

■ *If asked about HTTP vs internal protocol — describe what you did (POST for location mocking, GET for route fetching) without naming the protocol if you're unsure. You know the framework well enough.*

Q4: Appium End-to-End Navigation Tests

Interviewer question:

Walk me through what one of those 20 tests looked like end to end — what was the setup, what did Appium interact with, what was being asserted, and how did a regression manifest when caught?

Model Answer:

These 20 tests were our CAT2 automation suite — CAT0 being the most critical (start route, reroute — app is non-functional if these break), CAT1 being less severe, and CAT2 being the full breadth covering all apps and features. My tests covered: driver settings persisting after OTAs, user preferences, charger database checks, and satellite vs vector map mode.

Appium interacted with the navigation app by locating elements via resource IDs primarily, with XPath as a fallback. Flakiness was a real issue with nightly runs — we mitigated it with retry-on-failure logic and proper implicit waits in our base test functions to handle timing inconsistencies.

Since these ran on main and hotfix branches nightly rather than on MRs, they caught issues that slipped through earlier MR-level checks — things like text misalignment, missing data, or slow load times that only

surfaced in fuller integration scenarios. More than 15 regressions were caught historically across the suite during active feature development.

Education — Bonus Answers

Q: Where did your coursework actually help vs. self-teaching?

Model Answer:

School gave me the mathematical foundations — linear algebra for vector spaces and matrix manipulation, DSA for algorithm design, OOP in C/C++ for systems thinking. But most of the applied depth I needed — ROS2, PyTorch, spconv, projective geometry in code — I picked up independently through projects. The gap between school and the real world was in systems knowledge: how to structure a ROS2 node, debug a broken point cloud pipeline, read a research paper and turn it into working code. I closed that by building things. Interestingly, when the course material eventually caught up, it clicked faster because I'd already wrestled with it in practice.

Q: Linear algebra in the voxel pipeline — be specific (revisit in WATonomous section)

Model Answer:

The most direct place linear algebra showed up was in the depth-to-point-cloud conversion. Every pixel in the depth image is a 2D coordinate and a distance value — to turn that into a real 3D point in camera space, you apply the pinhole camera model: $x = (u - cx) * z / fx$, $y = (v - cy) * z / fy$. That's projective geometry — unprojecting from image space back into 3D using the camera intrinsics matrix K . I pulled fx , fy , cx , cy from the CameraInfo topic and applied that transform vectorized across the whole image using NumPy meshgrids, computing the entire point cloud in one shot instead of looping pixel by pixel.